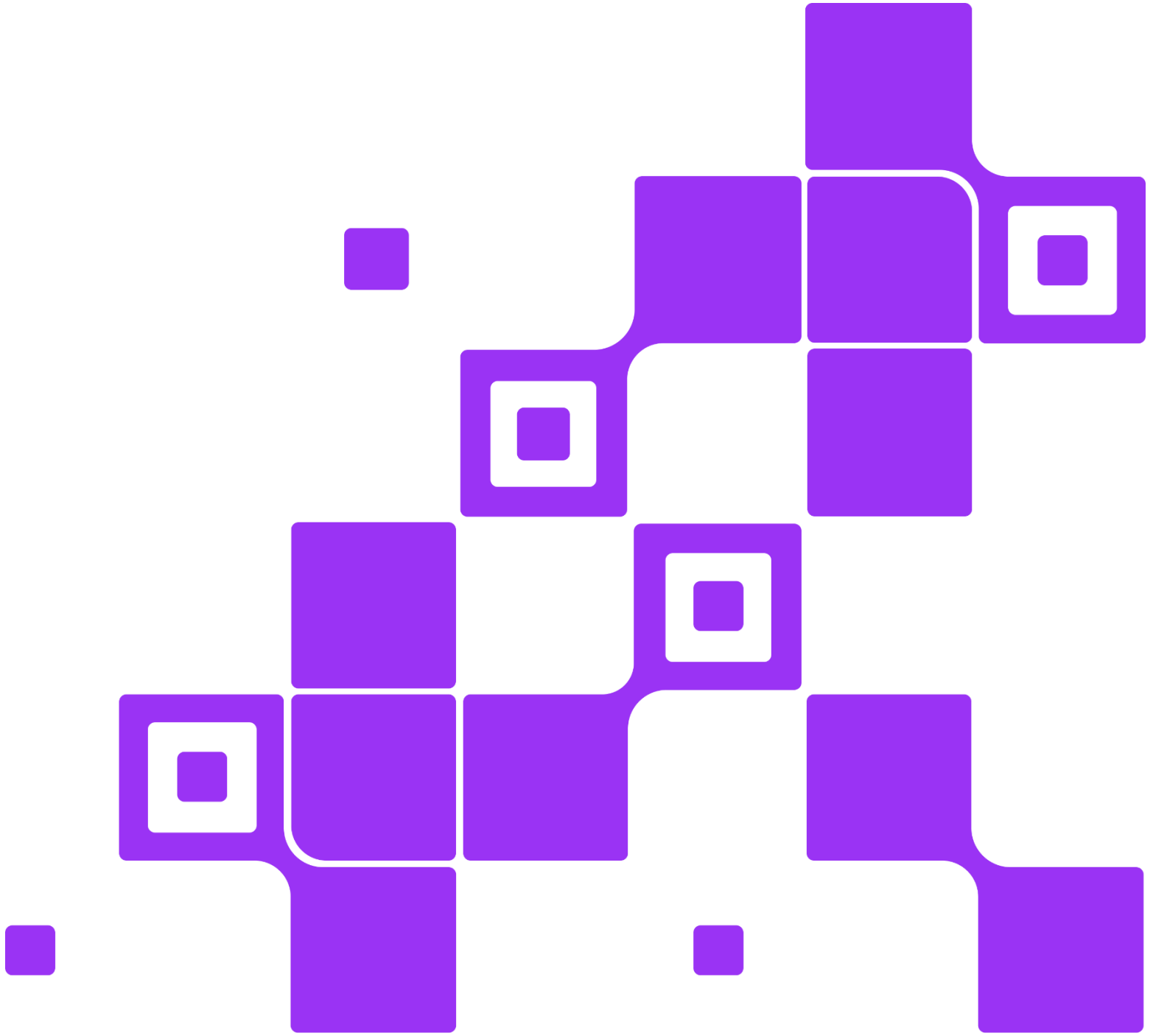
Iterator
ListIterator
Comparator
Comparable



Итератор (Iterator)

IThub

interface Iterator<E>

- итератор позволяет циклически проходить по коллекции, получая или удаляя элементы

interface ListIterator<E> extends Iterator<E>

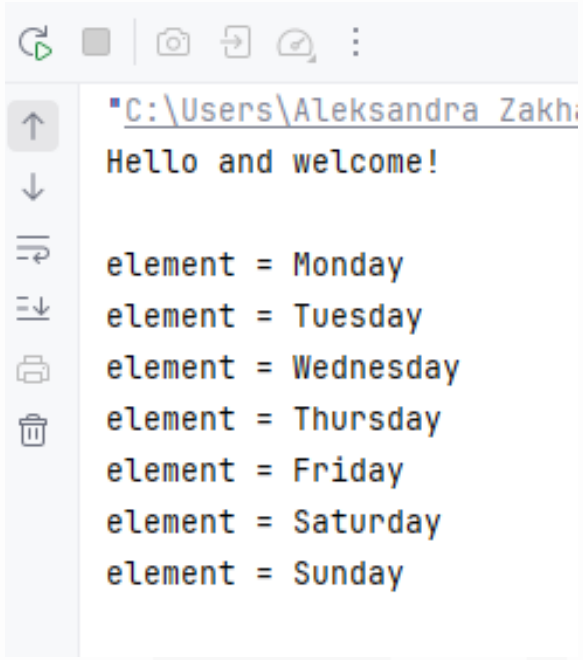
- интерфейс ListIterator расширяет Iterator с целью обеспечения двунаправленного обхода списка и модификации элементов

Использование итератора

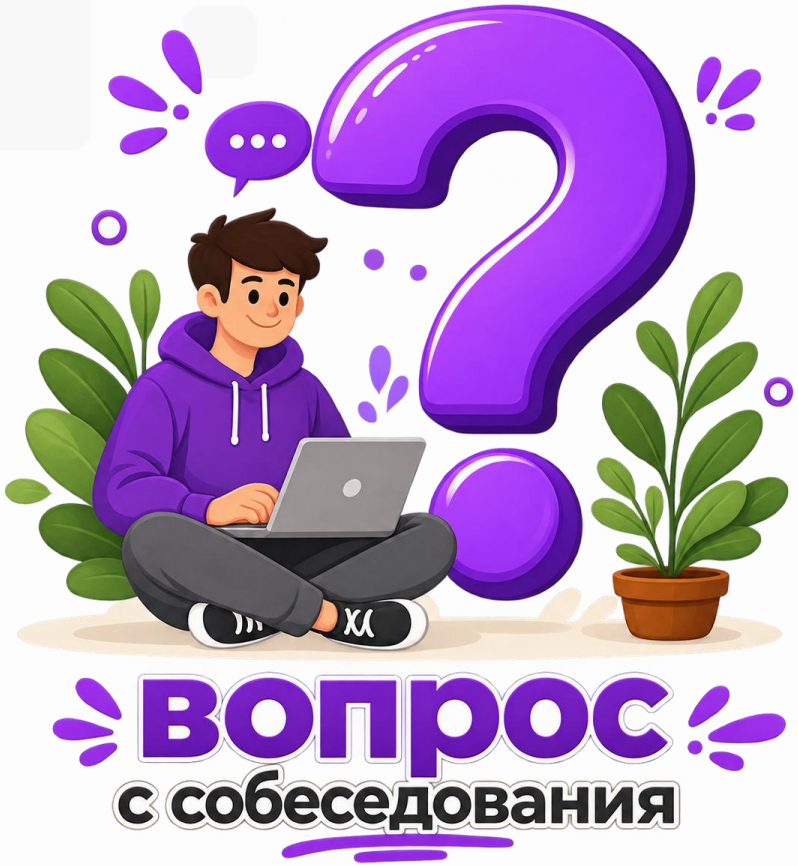
1. Получить итератор для начала коллекции, вызвав метод iterator() коллекции
2. Организовать цикл, внутри которого вызывается метод hasNext(). Обеспечить выполнение цикла, пока hasNext() возвращает true
3. Получать в цикле элементы с помощью вызова next()

```
ArrayList<String> list = new ArrayList<>();
list.add("Monday");
list.add("Tuesday");
list.add("Wednesday");
list.add("Thursday");
list.add("Friday");
list.add("Saturday");
list.add("Sunday");

Iterator<String> iterator = list.iterator();
while (iterator.hasNext()) {
    String day = iterator.next();
    System.out.println("element = " + day);
}
```



IThub



Что произойдет при вызове Iterator.next() без предварительного вызова Iterator.hasNext()?

Если итератор указывает на последний элемент коллекции, то возникнет исключение **NoSuchElementException**, иначе будет возвращен следующий элемент.

Зачем нужен Iterator, если есть for-each?

Мы можем сделать так:

```
for (String name : names) {
    System.out.println(name);
}
```

Или:

```
Iterator<String> iterator = names.iterator();

while (iterator.hasNext()) {
    String name = iterator.next();
    System.out.println(name);
}
```

На первый взгляд for-each удобнее. И действительно, для простого перебора чаще используют for-each.
Но у Iterator есть важная возможность — безопасно удалять элементы во время перебора.

IThub

Зачем нужен Iterator, если есть for-each?

```
List<String> names = new ArrayList<>(
    List.of(“Sasha”, “Anna”, “Ivan”, “Ivan”, “Masha”)
);

Iterator< String > iterator = names.iterator();

while (names.hasNext()) {
    String name = iterator.next();

    if (name.equals(“Ivan”)) {
        iterator.remove();
    }
}
```

Почему именно iterator.remove()?
Потому что итератор сам знает, какой элемент он только что вернул, и может корректно удалить его из коллекции.

IThub

Нельзя просто делать names.remove()
Например, вот так делать опасно:

```
Iterator<String> iterator = names.iterator();

while (iterator.hasNext()) {
    String name = iterator.next();

    if (name.equals("Ivan")) {
        names.remove(name);
    }
}
```

ConcurrentModificationException

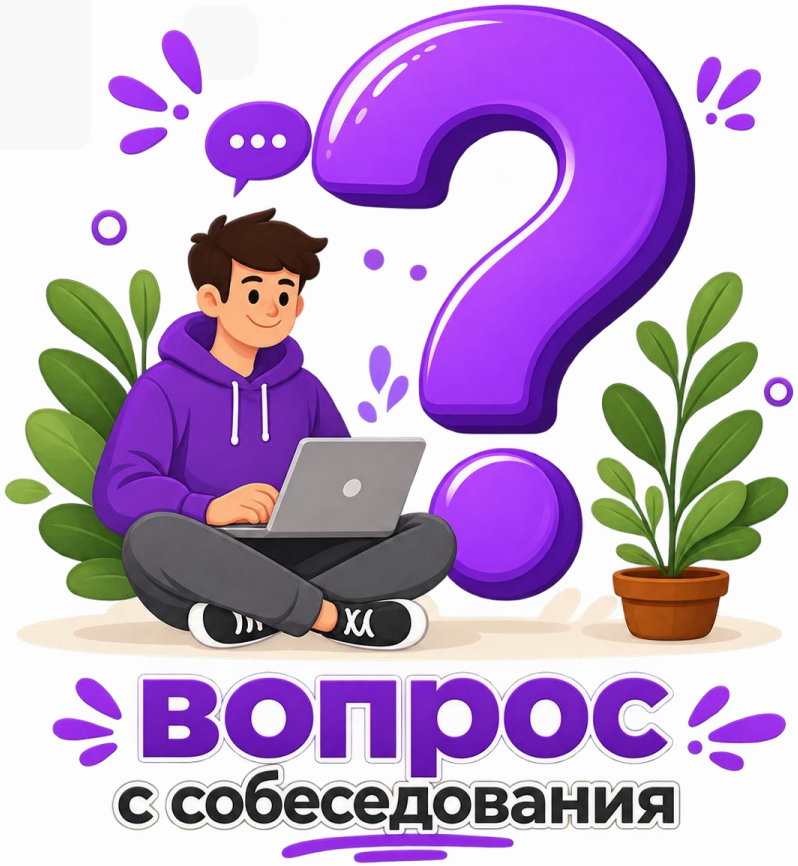
Правильно:

```
iterator.remove();
```

IThub

IThub

Сравните Iterator и ListIterator



- ListIterator расширяет интерфейс Iterator
- **ListIterator** может быть использован только для перебора элементов коллекции **List**
- Iterator позволяет перебирать элементы только в одном направлении, при помощи метода next(). Тогда как ListIterator позволяет перебирать список в обоих направлениях, при помощи методов next() и previous()
- ListIterator не указывает на конкретный элемент: его текущая позиция располагается между элементами, которые возвращают методы previous() и next()
- При помощи ListIterator вы можете модифицировать список, добавляя/удаляя элементы с помощью методов add() и remove(). Iterator не поддерживает данного функционала

Задача

IThub

Есть список:

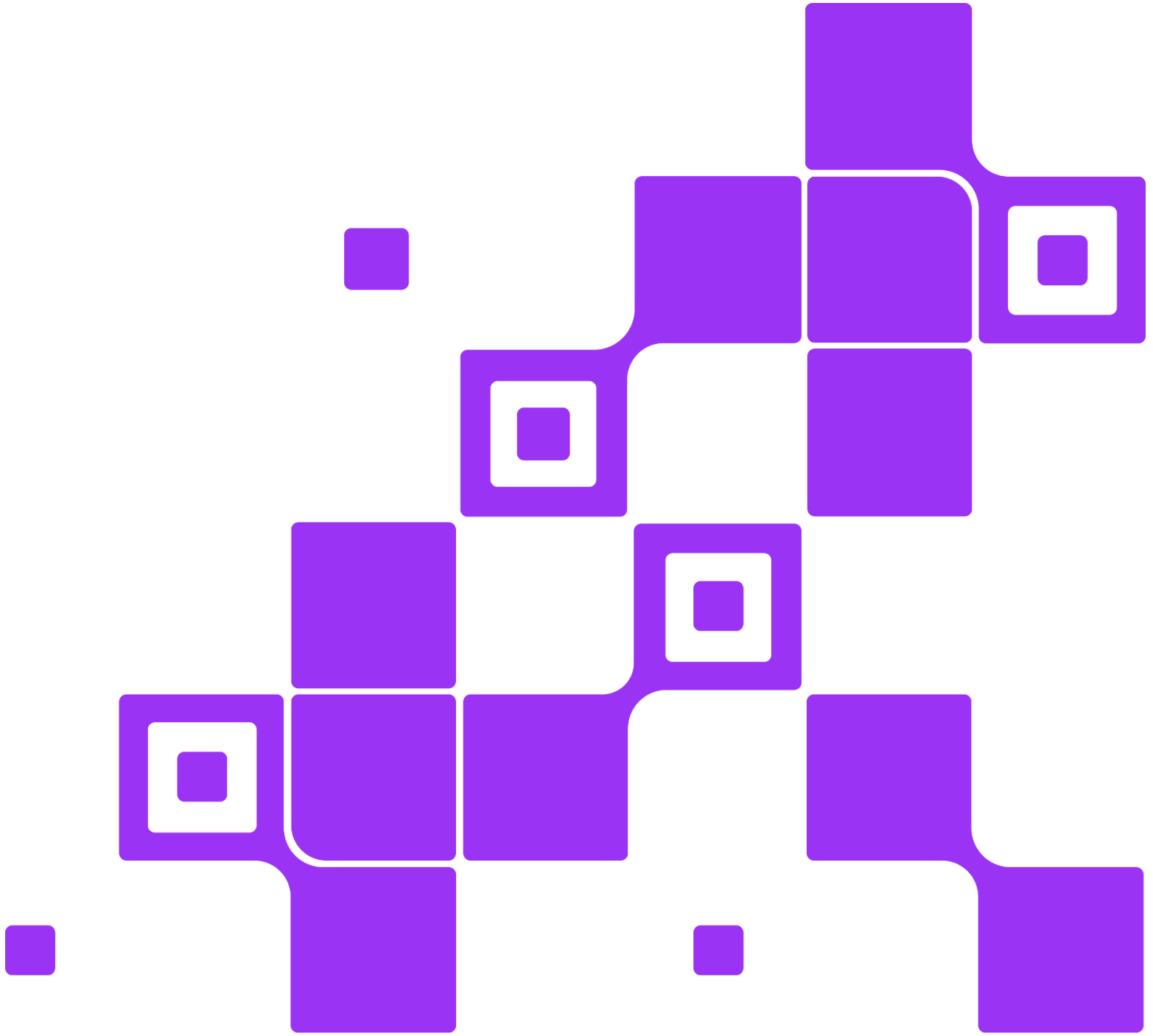
List<Integer> numbers = new ArrayList<>(List.of(1, 2, 3, 4, 5, 6, 7, 8));

Задача — с помощью Iterator удалить из списка все нечётные числа.

Используйте именно Iterator<Integer> и для удаления iterator.remove()

! Не используйте removeIf(), обычный for или stream().

Comparable
Comparator



Зачем вообще нужно сравнение объектов?

Допустим, у нас есть:

```
class Student {
    String name;
    int age;
}
```

```
List<Student> students = new ArrayList<>();
```

В нём: Alex 25, Bob 20, Kate 22

Если написать students.sort(...) Java должна понять: а по какому принципу сортировать студентов?

По имени? Alex Bob Kate

По возрасту? Bob 20 Kate 22 Alex 25

Поэтому нам нужен механизм сравнения.

Comparable находится в пакете java.lang.Comparable
У него один главный метод:

```
public interface Comparable<T> {
    int compareTo(T other);
}
```

IThub

Допустим, студентов по умолчанию мы хотим сортировать по возрасту:

```
class Student implements Comparable<Student> {

    private String name;
    private int age;

    public Student(String name, int age) {
        this.name = name;
        this.age = age;
    }

    @Override
    public int compareTo(Student other) {
        return Integer.compare(this.age, other.age);
    }

    @Override
    public String toString() {
        return name + " " + age;
    }
}
```

```
List<Student> students = new ArrayList<>();

students.add(new Student("Alex", 25));
students.add(new Student("Bob", 20));
students.add(new Student("Kate", 22));
```

```
Collections.sort(students);
```

```
Bob 20
Kate 22
Alex 25
```

Что означает результат compareTo()?

IThub

Метод compareTo()возвращает int.
Но нам не так важно конкретное число. Нас интересует знак:

- < 0 - первый объект меньше второго
- = 0 - объекты равны с точки зрения сравнения
- > 0 - первый объект больше второго

Но что если список студентов нужно сортировать по имени?
Вот здесь появляется Comparator.

Компаратор (Comparator)

IThub

interface Comparator<T>

- компаратор предоставляет возможность упорядочивания объектов по определенным правилам
- он определяет метод compare(), который принимает два объекта в качестве параметров и возвращает отрицательное число, ноль или положительное число в зависимости от того, какой объект должен идти раньше в порядке сортировки.

```
class Student {
    String name;
    int age;
    double grade;


    @Override
    public int compareTo(Student other) {
        return Integer.compare(this.age, other.age);
    }
}
```

Если нам нужно сортировать по age или grade?
Нам не хочется переписывать compareTo() каждый раз.

```
List<Student> studentsList = new ArrayList<>();
studentsList.add(new Student("Sasha“, 24, 5.0));
studentsList.add(new Student("Anna“, 22, 3.5));

System.out.println(studentsList);

Comparator< Student > compStudentName = new Comparator< Student >() {
        @Override
        public int compare(Student student1, Student student2) {
            return student1.getName().compareTo(student2.getName());
        }
    };

studentsList.sort(compStudentName);
System.out.println(studentsList);
```

ИЛИ

```
students.sort((s1, s2) ->
        s1.getName().compareTo(s2.getName())
);
```

ИЛИ

```
students.sort(
    Comparator.comparing(Student::getName)
);
```

Допустим, возраст по возрастанию
students.sort(Comparator.comparing(Student::getAge))

Получим:

Bob 20
Kate 22
Alex 25

По убыванию
students.sort(Comparator.comparing(Student::getAge).**reversed**());

Получим:

Alex 25
Kate 22
Bob 20

default Comparator<T> reversed()
возвращает компаратор с обратным порядком

Сортировка по нескольким полям

Допустим, сначала отсортируем по возрасту, а если возраст одинаковый — по имени:

```
students.sort(
    Comparator.comparing(Student::getAge)
            .thenComparing(Student::getName)
);
```

Сначала по возрасту по возрастанию, а внутри одинакового возраста — по имени в обратном порядке:

```
students.sort(
    Comparator.comparing(Student::getAge)
            .thenComparing(
                Student::getName,
                Comparator.reverseOrder()
            )
);
```

IThub

Null

Comparator может встретить null.

List<Student> students = ...и один студент имеет name = null;

Comparator.comparing(Student::getName) может привести к

проблеме при сравнении null.

Можно явно сказать, как работать с null:

```
Comparator.comparing(
    Student::getName,
    Comparator.nullsLast(Comparator.naturalOrder())
);
```

Теперь null будет в конце.
Есть и метод Comparator.nullsFirst(...) – null будет в начале

IThub

Comparable vs Comparator

	Comparable	Comparator
Где находится	java.lang	java.util
Главный метод	compareTo()	compare()
Кто определяет сравнение	Сам класс	Отдельный объект
Количество вариантов сортировки	Обычно один основной	Можно сколько угодно
Нужно изменять класс?	Да	Нет
Использование	Естественный порядок	Альтернативные порядки

Задание 1

Создай Comparator, который сортирует студентов:

1. Сначала по группе
2. Внутри группы — по средней оценке от большей к меньшей.
3. Если оценка одинаковая — по возрасту от меньшего к большему.
4. Если и возраст одинаковый — по имени в алфавитном порядке.

Задание 2

Создай Comparator, который сортирует студентов:

1. сначала по имени в обратном алфавитном порядке
2. если имена одинаковые — по оценке от большей к меньшей

Задание 3

Представь, что group может быть null.

Создай Comparator<Student> для сортировки только по группе если группа == null то студенты без группы должны оказаться в конце

```
public class Student {
    private String name;
    private int age;
    private double averageGrade;
    private String group;

    // конструктор с 4 аргументами
    // getters and setters
    // toString
}
```

```
List<Student> students = new ArrayList<>(List.of(
    new Student("Alex", 20, 4.5, "A"),
    new Student("Bob", 19, 3.8, "B"),
    new Student("Anna", 20, 4.9, "A"),
    new Student("Kate", 21, 4.2, "B"),
    new Student("Mike", 20, 4.5, "A"),
    new Student("John", 19, 4.9, "B"),
    new Student("Maria", 21, 3.8, "A")
));
```